Mizan.ai

Feature A (RAG) — Implementation Handoff for Coding Agent

This document is written directly to the coding agent (VS Code / Claude Code) implementing Feature A's RAG knowledge base. It specifies what to build, how to build it, which tools and libraries to use, and — critically — which parts of the existing, working codebase must NOT be touched. Read this document fully before writing any code.

July 2026 | Implementation handoff | Feature A: ZATCA/VAT RAG Knowledge Base

AGENT: Read Section 1 (“Do Not Touch”) before writing any code. If any task in this document appears to require modifying a file listed there, STOP and ask Mohammed before proceeding — do not work around it silently.

Task	Build the offline ingestion pipeline (local) and online inference pipeline (integrated with the existing FastAPI gateway + Modal endpoints) for Feature A's ZATCA/VAT RAG knowledge base.
Reference docs	RAG Data Source Strategy, Embeddings/Chunking/Pipeline Architecture, Pipeline Flowcharts (companion PDFs — the WHY; this document is the WHAT/HOW).
New code location	New, isolated module (e.g. feature_a/) — do not scatter changes across existing files beyond the minimal integration points in Section 7.
Do not deploy to Modal	This work does not create or modify any Modal app. The offline pipeline runs on Mohammed's local machine; the online pipeline is new logic in the existing FastAPI gateway calling already-deployed Modal endpoints.
Open decisions	Section 8 lists points not yet fully decided — ask Mohammed rather than guessing.

1. Do Not Touch — Working, Deployed Code

The following files/systems are already implemented, tested, and confirmed working in production or against real requests. None of this work requires modifying them. If a task seems to need a change here, stop and ask Mohammed — do not proceed on assumption.

File / System	What it is	Rule
modal_app.py, common/model_loader.py	Modal serving layer	All 4 endpoints (QwenBrain /generate, QwenLite /generate-lite, Translator /translate, Embedder /embed) are deployed and confirmed working end-to-end. Every fix in these files (T4 dtype, attention backend, tokenizer handling, etc.) was hard-won through real debugging — see the deployment lessons-learned doc. Do not redeploy, do not change GPU configs, do not touch model loading.
Auth system (register/login, JWT, bcrypt)	v3 handoff, implemented & tested	Confirmed done by Mohammed. Out of scope entirely.
Password reset flow	forgot-password / reset-password endpoints, Mailgun integration	Confirmed implemented and tested. Out of scope entirely.
persistence.py	Legacy activity-log & customer-memory tables	Leave exactly as-is. If Feature A needs its own persistent tables, add NEW tables/functions — do not modify the existing schema or functions in this file.
langgraph_chatbot.py	Homepage assistant / general chat (Feature D)	This is working: intent routing, dual-window memory (Postgres checkpointer + store), Arabic/English detection, the existing redirect-to-calculator/features behavior. Feature A is a NEW capability, not a rewrite of this file. See Section 7 for the one narrow, explicit integration point allowed here.
prompts.py	Homepage assistant system prompt	This is the general-assistant prompt (Feature D). Feature A needs its OWN, separate system prompt in a new file — do not overwrite or repurpose this one.

General rule: keep this work additive. New files in a new, clearly-named module. The only existing files that should change at all are the minimal integration points described in Section 7, and those changes should be small, additive, and reviewed carefully — not refactors.

2. Project Context (brief)

Mizan.ai is an Arabic-first AI platform helping Saudi SMEs with ZATCA/VAT compliance. Feature A is the ZATCA/VAT knowledge base and Q&A capability — a user asks a compliance question in Arabic or English, and the system answers grounded in official ZATCA/SOCPA source documents, with mandatory citations, version-awareness, and a strict fail-closed policy (never guess when unsure).

What already exists (build on this, don't duplicate it)

- FastAPI gateway (backend, currently has auth + password reset + the general chatbot route working)
- 4 Modal-hosted models, all confirmed working: QwenBrain (A10, agent brain / tool-calling), QwenLite (T4, chat/RAG — THIS is what Feature A's online pipeline calls for answer generation), Translator (MADLAD-400, T4), Embedder (BGE-M3, T4, dense 1024-dim — THIS is what the online pipeline calls to embed a user's query)
- LangGraph orchestration pattern already in use in langgraph_chatbot.py (StateGraph, Postgres checkpointer/store) — reuse this pattern for Feature A's own flow rather than inventing a different orchestration style
- Existing test-script convention: test_qwen_brain.py is a manual smoke-test script pattern (plain Python, requests, assert statements, print output) — follow this same style for Feature A's own test scripts, for consistency

What this handoff adds

- An **offline ingestion pipeline** (new, runs locally on Mohammed's machine, builds the knowledge base)
- An **online inference pipeline** (new logic, runs inside the existing FastAPI gateway process on request, calls existing Modal endpoints — does not add any new Modal deployment)

3. Tools & Libraries

3.1 — Offline pipeline (local machine only)

Library	Install	Used for
docling	pip install docling	PDF extraction — structured markdown/JSON, table structure preservation
FlagEmbedding + torch	pip install FlagEmbedding torch (with CUDA)	Local BGE-M3 embedding — MUST match the Modal deployment's config: use_fp16=True. Reuse load_embedding_model() / run_embedding() from the existing common/model_loader.py CODE (import or copy the functions) — do not reimplement embedding logic from scratch, to guarantee identical behavior to the Modal side.
qdrant-client	pip install qdrant-client	Local on-disk Qdrant for ingestion (QdrantClient(path=...))
anthropic	pip install anthropic	Claude API — table descriptions + cross-reference review. Requires ANTHROPIC_API_KEY (see Section 8 for provisioning).
pandas, openpyxl	pip install pandas openpyxl	Direct parsing of XLSX/CSV source files (Data Dictionary, code lists) — bypasses Docling entirely for already-structured files
PaddleOCR (+ EasyOCR fallback)	per the existing architecture doc	Arabic OCR quality, used by Docling internally where needed for RTL/mixed-script PDFs

3.2 — Online pipeline (FastAPI gateway)

- **No new Modal deployments.** Reuse the existing QwenLite (/generate-lite) and Embedder (/embed) endpoints exactly as they are.
- `qdrant-client` — the gateway process needs read access to the same Qdrant collection the offline pipeline writes to (see Section 8 for the deployment-location decision that's still open).
- `sqlite3` (standard library) — read access to the structured table store.
- `requests` — already used elsewhere in the codebase for calling Modal endpoints; reuse the same pattern.

4. Proposed Module Structure

Check the existing repository layout first and adapt naming to match convention. The important principle is isolation — all new code lives under one clearly-named module, not scattered across existing files.

```
feature_a/
  offline/
    config.py   # paths, Qdrant collection name, model config
    schema.py   # Qdrant collection + SQLite schema setup
    extract.py   # Docling wrapper + XLSX/CSV direct parse
    chunker.py   # structural chunking, article-level splits
    tables.py   # SQLite table storage + Claude description generation
    references.py   # regex + Claude cross-reference detection
    embed.py   # local BGE-M3 wrapper (reusing model_loader.py code)
    ingest.py   # CLI entrypoint: python -m feature_a.offline.ingest
  online/
    config.py
    retrieval.py   # Qdrant search, table/reference resolution, context cap
```

```
  fallback.py   # the two bilingual fallback messages + triggers
  answer_generation.py   # QwenLite call + Feature A's own system prompt
  pipeline.py   # orchestrates the online flow, called by the FastAPI route
requirements-offline.txt
requirements-online.txt
tests/
  test_ingestion_smoke.py
  test_retrieval_smoke.py
```

5. Offline Pipeline — Implementation Task List

Implements the full design from the Embeddings/Chunking/Pipeline Architecture document. Follow this order — later steps depend on earlier ones (in particular: all tables must be registered before cross-reference detection runs).

#	Task	Implementation detail
1	**Fetch**	CLI or script step: pull source PDFs/XLSX from the URLs cataloged in the RAG Data Source Strategy document. Save to a local raw/ folder tagged with fetch date. Handle missing/moved URLs gracefully (log and skip, don't crash the whole run).
2	**Type-detect & route**	By extension: .pdf → Docling path; .xlsx/.csv → pandas path; .xsd → copy to wherever the Feature E codebase expects reference schemas (confirm exact target path with Mohammed — Feature E isn't built yet, so this may just be a staging folder for now).
3	**Extract (Docling)**	Convert PDFs to structured markdown/JSON. Preserve table structure (do not flatten). Tag each extracted document with language (ar/en — known from the source URL path, not re-detected).
4	**Quality gate**	Flag any table with merged cells or dense numeric data (the penalty matrix and rate/exemption matrix are known examples) for manual/Claude review before it's trusted. On a detected extraction error: correct the data in the extracted output directly — do NOT re-run Docling expecting a different result, its merged-cell failure is systematic. Add a corrected:true flag + date to any manually-fixed table so it isn't redone on the next monthly run.
5	**Chunk (prose)**	One chunk per article. Regex-detect “Article N” (English) and “المادة N” (Arabic, handle Arabic-Indic digits too). Sub-split only if an article exceeds ~500 words. Prefix each chunk's text with an in-language context header (e.g. “VAT Implementing Regulations, Article 47:”) — embedded into the text itself, not just metadata. No overlap between chunks.
6	**Tables → SQLite**	Two tables: tables_registry (one row per table, with document_type, version_label, jurisdiction, language, dates, description, table_ref) and table_rows (the actual row data as JSON, FK to table_ref). Decide pointer granularity by row count: <~20 rows → one whole-table pointer; larger (Data Dictionary, code lists) → sharded, one pointer per row/logical group.
7	**Table descriptions (Claude)**	One short natural-language description per table (or per row-group for sharded large tables), generated once via the Claude API. This description is what gets embedded — not the table data itself.
8	**Detect cross-references**	Regex first pass for “Article N” citations (both languages). Claude review pass on top, to catch indirect/spelled-out references the regex misses. Classify each as type:article (needs document_type + article_number) or type:table (needs table_ref). Attach as a references list on the citing chunk. IMPORTANT: run this step only after ALL documents' tables are registered (step 6 complete across the whole corpus), so table references can resolve correctly.
9	**Embed (local BGE-M3)**	Embed every prose chunk's full text (including its header) and every table description. Use the exact same checkpoint/config as the Modal Embedder (use_fp16=True). Write a smoke test that embeds a fixed test string locally and confirms the output vector shape/values are consistent with what the Modal endpoint would produce for the same input, before trusting a full ingestion run.

#	Task	Implementation detail
10	**Write to Qdrant**	Upsert prose chunks (content_kind=prose) and table-pointer chunks (content_kind=table_pointer, with table_ref) with the full payload schema from Section 6 of the Architecture document (document_type, article_number/table_ref, jurisdiction, language, version_label, is_current, effective_start_date, effective_end_date, references, source_site, source_url, text).

6. Online Pipeline — Implementation Task List

Called from a new FastAPI route (see Section 7) on every Feature A chat request. Read-only against Qdrant + SQLite — this pipeline never writes to the knowledge base.

#	Task	Implementation detail
1	**Detect language**	Reuse the existing detect_language() logic/pattern from langgraph_chatbot.py (Arabic-script regex check) — don't reimplement differently.
2	**Non-KSA country check**	A simple, editable keyword list (English + Arabic country names: Bahrain/البحرين, UAE/الإمارات, Oman/عمان, Qatar/قطر, Kuwait/الكويت). If matched → return the wrong-jurisdiction fallback immediately, skip retrieval entirely. Keep this list in config.py, not hardcoded inline, so Mohammed can edit it without touching logic.
3	**Embed query**	POST to the existing Modal Embedder /embed endpoint. Handle timeout/cold-start the same way langgraph_chatbot.py's _generate_reply() already does (reuse the _warming_up pattern if there's a user-facing “warming up” state to show).
4	**Search Qdrant (same-language-first)**	Filter language = detected_query_language. If results are empty or below a relevance threshold (define one — flag to Mohammed if not specified), retry once WITHOUT the language filter.
5	**Nothing found?**	If still nothing sufficiently relevant after the retry → return the single “nothing found” fallback message (exact bilingual text in Section 7.3 of this document — hardcode both, select by detected query language).
6	**Resolve table pointers**	For any retrieved chunk with content_kind=table_pointer, look up table_ref in SQLite and fetch the relevant row(s) only — not the whole table for large sharded tables.
7	**Resolve cross-references (one hop)**	For each retrieved chunk's references list: type=article → Qdrant lookup using (document_type, article_number, and the CITING chunk's own version_label — inherited, not stored on the reference itself); type=table → SQLite lookup by table_ref. Do not chase a second hop.
8	**Apply context cap**	Cap total chunks (original retrieval + resolved references/tables) at 6. If the cap is hit, keep the original retrieval matches and drop resolved references first.
9	**Cross-language wrapper check**	If the final answer content came from the cross-language fallback (step 4's retry), wrap it in a same-language explanatory message and present the retrieved content UNTRANSLATED — do not call the Translator/MADLAD endpoint here. NOTE: exact wording for this wrapper message was NOT finalized during design (unlike the two fallbacks in Section 7.3) — draft it in the same tone/style as those two and confirm with Mohammed before shipping. See Section 8, item 13.
10	**Generate answer (QwenLite)**	POST to the existing Modal QwenLite /generate-lite endpoint. Feature A needs its OWN system prompt (new file, do not touch prompts.py) enforcing: default to is_current=true content; explicitly label any non-current content used; mandatory citation on every substantive claim (source_site + jurisdiction + version_label + document_type); never fabricate a citation.
11	**Return response**	Answer + structured citations, or one of the two fallback messages. Match the existing response shape used by langgraph_chatbot.py's endpoint where reasonable, for frontend consistency.

Conversation history: reuse the existing in-memory / Postgres-store pattern from langgraph_chatbot.py. Keep only (user query, final response) pairs, capped at 8 turns — same shape as the existing dual-window memory, not a new memory system.

7. Integration Points With Existing Code

These are the ONLY places existing files should change. Keep each change small and additive.

7.1 — New FastAPI route

Add a new route (e.g. `POST /api/chat/kb` or match existing naming convention — check the gateway's current route file) that calls `feature_a.online.pipeline`. This is a NEW route, not a modification of the existing chat route.

7.2 — langgraph_chatbot.py redirect target

The existing chatbot already detects ZATCA/VAT intent (classify_intent()) and returns a static redirect suggestion (fallback_reply() for intent == “calculator”, and the general “not ZATCA questions” behavior per the standing product constraint). **Ask Mohammed whether Feature A should now be wired as the actual handler for VAT-intent messages inside this existing flow, or whether it stays a fully separate UI entry point.** This is explicitly not decided — do not silently choose one. If wiring it in, the change should be a small, additive branch, not a rewrite of _generate_reply() or the intent classifier.

Do not modify classify_intent(), detect_language(), build_system_prompt(), or the memory/persistence functions in langgraph_chatbot.py. If Feature A needs something from this file, import and reuse it — don't edit it in place.

7.3 — Fallback messages (exact text, hardcode both)

These are the exact, final strings — do not paraphrase or regenerate them. Store both in `feature_a/online/fallback.py` and select by detected query language.

Wrong jurisdiction — English
Mizan.ai currently supports Saudi Arabia (ZATCA/KSA) tax and compliance guidance only. We don't yet cover other GCC countries. If you need support for [detected country], please reach out to our team — we're always looking to understand where to expand next.
Wrong jurisdiction — Arabic
بعد. إذا كنت بحاجة إلى دعم لـ الدولة المكتشفة، يرجى التواصل مع فريقنا - نحن نتطلع دائما إلى معرفة أين نوسع خدماتنا. الخاصة بالمملكة العربية السعودية هيئة الزكاة والضريبة والجمارك فقط، ولا يغطي دول مجلس التعاون الخليجي الأخرى يدعم . حاليا الإرشادات الضريبية والامتثال
Nothing found — English
I wasn't able to find a clear answer to this in Mizan.ai's current knowledge base. Rather than guess, I'd rather be upfront that this isn't covered with enough detail yet. For help with this specific question, please reach out to our team.
Nothing found — Arabic
الموضوع غير مغطى بتفصيل كاف حتى الآن. للحصول على المساعدة بخصوص هذا السؤال تحديدا، يرجى التواصل مع فريقنا. العثور على إجابة واضحة لهذا السؤال ضمن قاعدة معارف . الحالية. بدلا من التخمين، أفضل أن أكون واضحا بأن هذا لم أتمكن من

`[detected country]` in the wrong-jurisdiction message should be substituted with the actual country name matched in step 2 of the online task list (Section 6), in the same language as the surrounding message.

8. Open Decisions — Ask Mohammed, Don't Assume

These points were not fully settled during design and need a decision (or at least an explicit “default is fine” confirmation) before or during implementation. This list was compiled by reviewing the full design discussion for gaps — flag any of these back to Mohammed rather than picking silently, consistent with the fail-closed philosophy used throughout this design.

1. **Qdrant deployment location for production.** Design assumed local on-disk Qdrant for offline ingestion. Where does the ONLINE pipeline's Qdrant instance live — same host as the FastAPI gateway, a separate managed Qdrant Cloud instance, or self-hosted on the eventual GCE/GKE infrastructure? This affects how the gateway connects to it (path vs URL) and needs deciding before the online pipeline can be finished, even though the offline pipeline can be built and tested locally now.
2. **ANTHROPIC_API_KEY provisioning.** The offline pipeline needs this as a new secret, separate from the existing Modal secrets (huggingface-secret) and the auth/email secrets. Confirm where this should live (local .env for now; GCP Secret Manager later per the standing infra plan).
3. **Source URL completeness.** The Data Source Strategy document's URL catalog was correct as of research time, but ZATCA pages can move. Before the first full ingestion run, verify each URL still resolves — don't assume the catalog is still 100% accurate without a check.
4. **Local GPU environment setup.** Confirm torch is installed with CUDA support (not CPU-only) on Mohammed's machine before running the offline embedding step at scale — a silent CPU fallback would still work but be much slower, and might mask a config problem worth catching early.
5. **Start small.** Recommend running the full offline pipeline against 1–2 sample documents first (not all ~11 categories at once) to validate every stage end-to-end — including a real Claude API call for one real table — before committing to a full ingestion run and its associated Claude API cost.
6. **Claude API cost/rate awareness.** No budget or rate limit was specified for the table-description and cross-reference review calls. Implement with a simple running call-counter logged to console, and do not implement unbounded retry loops on API failure — fail a single table/document and move to the next, logging what failed, rather than looping indefinitely.
7. **Docling's real-world behavior may not exactly match the design's assumptions.** Run a real extraction test against one real ZATCA PDF early. If the auto-detected article/table structure looks meaningfully different from what the chunking design expects (e.g. Docling's markdown headers don't cleanly align with “Article N” patterns), report this back rather than forcing the mismatch silently into the wrong chunk shape.
8. **Relevance threshold for “nothing found.”** The design says “if nothing sufficiently relevant comes back” but doesn't specify a numeric similarity cutoff. Propose a starting threshold, note it clearly in code as a tunable constant, and flag it to Mohammed as something to calibrate once real queries are being tested — don't bury it as a magic number.
9. **Staged writes vs. direct writes to the live Qdrant collection.** If an ingestion run fails partway through, does it leave the knowledge base in an inconsistent state? Not explicitly decided. Recommended (but confirm with Mohammed): ingest into a new/staging collection, verify it, then swap it in as the live collection — rather than writing directly into the collection that's currently serving the online pipeline.
10. **Where retrieval logic actually runs.** The original broader architecture docs describe an MCP-server-per-domain pattern. This design work treated the online pipeline as logic inside the FastAPI gateway. Confirm with Mohammed whether Feature A's retrieval should eventually be its own MCP server (matching that original plan) or stay as gateway-internal logic for v1 — don't assume either way.
11. **Feature A ↔ existing chatbot wiring** (also listed in Section 7.2) — repeated here because it's a genuine open product decision, not just a code detail.
12. **Logging/observability.** The broader stack plans LangSmith (dev) and Langfuse (prod, self-hosted). Confirm whether Feature A's pipeline should emit to these from day one, or whether basic Python logging is acceptable for this first build.
13. **Cross-language wrapper message wording.** Unlike the two fallback messages in Section 7.3 (finalized, exact text), the same-language explanatory wrapper shown when an answer falls back to the other language's

content (online task 9) was discussed conceptually but its exact bilingual wording was never finalized. Draft it in the same tone as the Section 7.3 messages and confirm with Mohammed before shipping — do not invent final copy unreviewed.

9. Testing & Acceptance Criteria

9.1 — Offline pipeline

- Run ingestion against exactly one sample document end-to-end first.
- Verify Qdrant received the expected chunk count, and spot-check payload fields (document_type, article_number, language, version_label, is_current, references) against what the source document actually contains.
- Verify SQLite table_rows contains correct data for at least one table, checked by hand against the source PDF.
- Run one manual test query and confirm it retrieves the chunk you'd expect a human to pick.

9.2 — Online pipeline

Follow the style of the existing test_qwen_brain.py (plain Python script, requests, assert statements, readable print output) for a new test_feature_a_pipeline.py covering:

- A plain in-scope Arabic query — expect a grounded answer with full citation.
- The same question in English — expect a grounded answer with full citation, matching content.
- A query naming a non-KSA GCC country (e.g. “Bahrain VAT rate”) — expect the wrong-jurisdiction fallback, no retrieval attempted.
- A query clearly outside all covered topics — expect the “nothing found” fallback.
- A query that should resolve to a table (e.g. a penalty amount) — expect the answer to reflect the resolved SQLite row, not just the embedded description text.
- A query that should trigger one-hop cross-reference resolution — expect the referenced article's content to be reflected in the answer.

10. Final Reminders

- This document is the WHAT/HOW. The companion documents (RAG Data Source Strategy, Embeddings/Chunking/Pipeline Architecture, Pipeline Flowcharts) are the WHY — refer back to them for reasoning behind any design choice that seems non-obvious.
- If any instruction here conflicts with what's already deployed and working, STOP and ask Mohammed — do not silently resolve the conflict in either direction.
- No new Modal app or endpoint is created as part of this work.
- Keep the diff scoped to the new feature_a/ module plus the specific, narrow integration points in Section 7. Avoid opportunistic refactors of unrelated files, even if you notice something that could be improved — flag it separately instead.
- When in doubt between two reasonable implementations of something this document leaves slightly open, prefer the option that fails closed (returns a fallback / asks for clarification) over one that guesses — this is the consistent design philosophy across the whole system.

Feature A implementation handoff — Mizan.ai. Companion to the RAG Data Source Strategy, Embeddings/Chunking/Pipeline Architecture, and Pipeline Flowcharts documents.